

Input: $\mathbf{D}_{src}$, source GPUs; $\mathbf{D}_{tgt}$ target GPUs; $\mathbf{n}$: number of GPUs to scale;
 $\mathbf{BW}_i$: return the scale out network bandwidth of GPU i ;
 $\mathbf{L}_i$: return the leaf switch ID of GPU i ;
 $\mathbf{Scale}_i$: the scaleup domain ID of GPU i .
Output: $\mathbf{Plan}$, a graph indicating how the parameters are loaded.

```
1  $\mathbf{D}_{src} = \mathbf{D}_{src}.prune().group\_by(\mathbf{L}_i).sorted\_by(sum([\mathbf{BW}_i] )).flatten()$ 
2  $\mathbf{D}_{tgt} = \mathbf{D}_{tgt}.group\_by(\mathbf{Scale}_i).sorted\_by(sum([\mathbf{BW}_i] ))$ 
     $\triangleright$  sort  $\mathbf{D}_{tgt}$  according to the order of Leaf ID in  $\mathbf{D}_{src}$ 
     $.groupby(\mathbf{L}_i).sortby(\mathbf{D}_{src}.index(\mathbf{L}_i).min()).flatten()$ 
3  $\mathbf{m} = 0; \mathbf{Plan} = \emptyset$ 
4 While not  $\mathbf{D}_{tgt}.empty()$  and  $\mathbf{m} < \mathbf{n}$ :
     $\triangleright \mathbf{G}_{tgt}$ : targets group connected by the scale up network
     $\mathbf{L}_{tgt}$ : the leaf switch of these targets.
5  $\mathbf{G}_{tgt}, \mathbf{L}_{tgt} = \mathbf{D}_{tgt}.pop\_front()$ 
6 If  $\mathbf{D}_{src}.filter(= \mathbf{L}_{tgt}).sum([\mathbf{BW}_i] ) \geq \mathbf{G}_{tgt}.sum([\mathbf{BW}_i] )$ 
     $\triangleright$  sources within the leaf have sufficient bandwidth for loading
7  $\mathbf{D}_{tmp} = \mathbf{D}_{src}.truncate(= \mathbf{L}_{tgt}); \mathbf{D}_{src} ++ \mathbf{D}_{tmp}$ 
8  $\mathbf{G}_{src} = \mathbf{D}_{src}.truncate(sum([\mathbf{BW}_i] ), \geq \mathbf{G}_{tgt}.sum([\mathbf{BW}_i] ))$ 
9  $\mathbf{Plan} = \mathbf{Plan} ++ (\mathbf{G}_{src}, \mathbf{G}_{tgt}); \mathbf{m} = \mathbf{m} + |\mathbf{G}_{tgt}|$ 
10  $\mathbf{D}_{src} = \mathbf{G}_{tgt} ++ \mathbf{D}_{src}$ 
11 return  $\mathbf{Plan}$ 
```

Notation:

- $++$ $\triangleright$ join 2 data collections, e.g., $[x, y] ++ [z, w] = [x, y, z, w]$
- $[\mathbf{Func}]_i$ $\triangleright$ apply $\mathbf{Func}$ to all elements of a data collection, e.g.,
 $\mathbf{G}.sum([\mathbf{BW}_i])$ is the summation of bandwidth of all elements in $\mathbf{G}$
- $\mathbf{Iter}.index(\mathbf{value})$ $\triangleright$ locations where a unique value first occurs in $\mathbf{Iter}$
- $\mathbf{Iter}.truncate(\mathbf{predicate})$
 $\triangleright$ pop front until the first remaining element satisfies predicate
- $\mathbf{Iter}.truncate(flatMap, \mathbf{predicate})$
 $\triangleright$ pop front and apply flatMap onto the popped elements
until the result satisfies predicate

